

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 1

MODEL SELECTION AND HYPERPARAMETER TUNING

Reading Time	50 minutes
Prior Knowledge	Machine learning
Keywords	Hyperparameter, grid search, random search, Cross validation

In the previous modules, we have trained and evaluated the performance of machine learning models. Machine learning models have a number of parameters that the data learn; model parameters are found by fitting our model on the training dataset. Hyperparameters on the other hand cannot be learned by our normal training process and are fixed before we begin the training process. In this module, we will learn more about hyperparameters. In this lesson, we will begin by discussing how to select the best performing model and the various hyperparameter tuning strategies.

1. Improving Model Performance

Before putting our machine learning model(s) to production, we should make sure that the model gives the best possible solution devoid of bias and can generalize well on new unseen data.

We will cover the following topics in this section.

- Overfitting and underfitting.
- Train, validation, and test sets.
- Cross-validation.

We begin with data splitting into train, test, and validation sets.

1.1 Train, validation and tests sets

It is advisable to split the data into training, testing, and validation sets especially if we want to tune our hyperparameters. The validation set helps us get an unbiased evaluation of our model performance and can therefore tune our model hyperparameters before passing the model to the test dataset.

Given a dataset, we can get the train, validation, and test set by using one of the following:

- Random Split
- Stratified Split

Random split involves splitting the original data into pre-defined proportions representing the train, validation, and test sets by first shuffling the data so as to overcome the challenge of there existing a pattern based on the data indices.

Stratified split splits the train, validation, and test sets according to the proportion of samples in the target variable. Stratified split is often used when our data is imbalanced.

Let us see an example of this using the California housing dataset whose target variable is the house price median.

```
In [ ]: from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X, y = housing.data, housing.target * 10
```

```
In [ ]: X.head()
```

```
In [ ]: y.head()
```

We will use the decision tree regressor to predict the median house price with an interest in getting the generalization of our model. So, we start by training the model on the entire dataset.

```
In [ ]: from sklearn.tree import DecisionTreeRegressor

regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X, y)
```

We also evaluate the model on the dataset to see how it performs.

```
In [ ]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(X)
score = mean_absolute_error(y, target_predicted)
print(f"On average, our regressor makes an error of {score:.3f} ")
```

The resulting mean absolute error is actually very impressive: it says that our model actually got the median price of each house accurately. So, the question is should we pick this model? The simple answer is not so fast! Remember we trained and evaluated our model on the same dataset and the decision tree regressor could have memorized the dataset and would therefore give us the wrong notion about its ability to generalize on unseen data.

When training our model, we are interested in it minimizing the error on unseen data, and this is the idea behind us splitting the data into train and test sets.

So, we can begin by splitting our dataset as shown below. Again, we train our model on the training set and evaluate its performance on the same data and record the error.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

We again train the model on our training data.

```
In [ ]: regressor.fit(X_train, y_train)
```

We predict the data on our training data and look at the resulting training error.

```
In [ ]: y_predicted = regressor.predict(X_train)
score = mean_absolute_error(y_train, y_predicted)
print(f"The training error of our model is {score:.3f}")
```

The output tells us that the model was able to accurately predict the actual house value. To check the validity of the model's strength, we use the test data, which has not been 'seen' by the model, to predict the house median price.

```
In [ ]: y_predicted = regressor.predict(X_test)
score = mean_absolute_error(y_test, y_predicted)
print(f"The testing error of our model is {score:.3f} ")
```

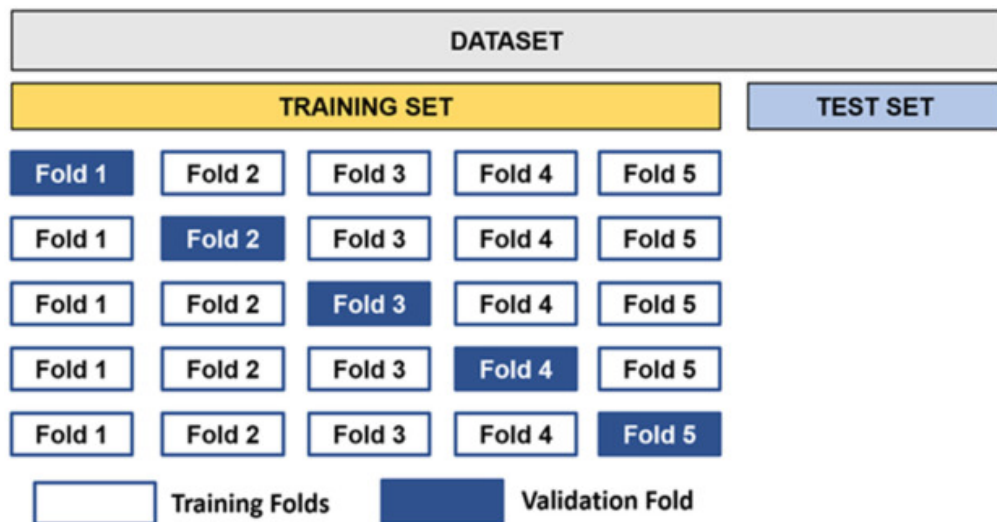
We now see that there is a bigger error on the testing dataset as compared to the training set and hence signs that our model is *overfitting*. To avoid issues with overfitting, we can use cross-validation that splits the data into smaller chunks iteratively so that it covers the entire train data, which is alternatively split into train and validation sets.

1.2 Cross-Validation

In this technique, the train data is split into train and validation sets multiple times, and each split is called a **Fold**. In this subsection, we will discuss the k-fold cross-validation.

K-fold cross-validation splits the data into k segments where $k-1$ segments are used for training and the remaining segment used for validation as shown in the diagram below.

Fig 1: K-Fold Cross Validation



Source: Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.

Below are the steps involved in performing a cross-validation:

1. Shuffle the dataset.
2. Hold out the test data.
3. Perform k-fold cross validation on the train dataset.
4. Get the best evaluation score, which is the mean of all the folds.
5. Perform model evaluation on the test data.

Cross-validation allows us to estimate the robustness of our model by splitting the dataset repetitively a given number of times.

The next cells demonstrate how to apply cross-validation on the dataset.

```
In [ ]: from sklearn.model_selection import ShuffleSplit, cross_validate

cv = ShuffleSplit(n_splits=40, test_size=0.3, random_state=0)
cv_results = cross_validate(regressor, X, y, cv=cv, scoring="neg_mean_absolute_e
```

We convert the `cv_results` Python dictionary into a dataframe for easier visualization.

```
In [ ]: import pandas as pd

cv_results = pd.DataFrame(cv_results)
cv_results.head()
```

We negate the `test_score` to get the actual test errors because the parameter `scoring="neg_mean_absolute_error"` gives negative values of the mean absolute error.

```
In [ ]: cv_results["test_error"] = -cv_results["test_score"]
```

```
In [ ]: cv_results.head(10)
```

The plot below shows how the testing errors are distributed.

```
In [ ]: import matplotlib.pyplot as plt

plt.hist(cv_results["test_error"], density=True)
# cv_results["test_error"].plot.hist(bins=10, edgecolor="black")
plt.xlabel("Mean absolute error")
plt.ylabel("Frequency Percentage %")
plt.suptitle(
    "Fig. 2: Test Errors Distribution.", fontweight="bold", horizontalalignment=
)
plt.show()
```

```
In [ ]: print(
    f"The mean cross-validated testing error is: "
    f"{cv_results['test_error'].mean():.3f}"
)
```

The error has reduced when compared to the initial error without using cross-validation.

```
In [ ]: print(
    f"The standard deviation of the testing error is: "
    f"{cv_results['test_error'].std():.3f}"
)
```

Cross-validation reduces the error on our model albeit at a smaller fraction as compared to the case when we did not use cross-validation.

A problem tends to occur in our bid to improve model performance: we can find ourselves solving one problem (underfitting) while introducing another problem (overfitting). In the next subsection, let's discuss the overfitting and underfitting concepts.

1.3 Overfitting and Underfitting

When the model memorizes the train data, then we say overfitting has occurred. In such a case, we get a high evaluation score on our train dataset but a not-so-impressive score on the test data set; therefore, the model fails to generalize well on unseen data.

Models that overfit respond to random noise in the training set and thus become sensitive to small fluctuations in the data. Overfitting is a combination of low bias and high variance.

Underfitting on the other hand is characterized by a low performance score on both the training dataset and the validation dataset. Underfitting is caused by simple algorithms that are unable to learn complex patterns existing in our data. Underfitting is also referred to as high bias and often low variance.

A right fit model neither underfits nor overfits, and the model performance in the training set is comparable to the test set. Linear machine learning algorithms such as logistic regression tend to underfit on the dataset while non-linear models like polynomial regression tend to overfit a dataset.

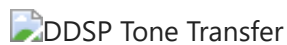
In case our model underfits, to improve its performance, we can

1. Use non-linear algorithms.
2. Increase the complexity of the model by tweaking its parameters.
3. Use non-parameterized algorithms.

To curb overfitting of our models, we

1. Increase the data size used for training to improve the generalizing ability of the model.
2. Use regularization techniques like the L1 and L2 norms.
3. Tune the hyperparameters of our model.
4. Reduce the number of features in our dataset.
5. Reduce the complexity of the model.
6. Apply cross-validation techniques.

Fig 3: Overfitting and Underfitting Examples



Source: [Badillo, Solveig, et al. "An Introduction to Machine Learning." *Clinical Pharmacology & Therapeutics*, vol. 107, no. 4, 2020, pp. 871-885.](#)

In the next cells, we evaluate our model and determine whether it is a good fit, underfitting or overfitting on our dataset.

```
In [ ]: import pandas as pd
        from sklearn.model_selection import ShuffleSplit, cross_validate

        cv = ShuffleSplit(n_splits=30, test_size=0.2)
        cv_results = cross_validate(
            regressor,
            X,
            y,
            cv=cv,
            scoring="neg_mean_absolute_error",
            return_train_score=True,
            n_jobs=2,
        )
        cv_results = pd.DataFrame(cv_results)
```

As discussed in the previous subsection, we negate the scores to get the actual train and test errors.

```
In [ ]: scores = pd.DataFrame()
        scores[["train error", "test error"]] = -cv_results[["train_score", "test_score"]]
```

The next cell plots the distribution of the errors.

```
In [ ]: import matplotlib.pyplot as plt

        scores.plot.hist(bins=50, edgecolor="black")
        plt.xlabel("Mean absolute error")
        plt.suptitle(
            "Fig. 4: Train and Test Errors Distribution via Cross-validation.",
            fontweight="bold",
```

```
horizontalalignment="right",
)
plt.show()
```

We can observe that the training error is zero, implying that our model is not underfitting. The test error is larger, and therefore, our model seems to have memorized the noise in our dataset. Hence, its generalization abilities have been compromised.

One of the challenges with hyperparameter tuning is that we can easily shift from a case of a model underfitting into another case of the model overfitting on our dataset. To overcome such challenges, we need to consider **validation curves**.

1.4 Validation Curves

The validation curve varies the values of the hyperparameter(s) and gives the intuition of how the model behaves given different hyperparameter values.

```
In [ ]: from sklearn.model_selection import validation_curve

max_depth = [1, 3, 5, 10, 15, 20, 25, 30]
train_scores, test_scores = validation_curve(
    regressor,
    X,
    y,
    param_name="max_depth",
    param_range=max_depth,
    cv=cv,
    scoring="neg_mean_absolute_error",
    n_jobs=2,
)
train_errors, test_errors = -train_scores, -test_scores
```

In the example, we will restrict ourselves to one of the hyperparameters, the `max_depth`.

```
In [ ]: plt.plot(max_depth, train_errors.mean(axis=1), label="Training error")
plt.plot(max_depth, test_errors.mean(axis=1), label="Testing error")
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 5: Validation Curve for Decision Tree",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

From the above plot, we see that for lower values of maximum depth, the model seems to underfit while as the maximum depth of trees increases, the model overfits on our dataset. So, for the values of 10, we can say that our model will provide a good fit and hence better generalization on an unseen dataset. The plot below repeats the above plot but on training errors.

```
In [ ]: plt.errorbar(
    max_depth,
    train_errors.mean(axis=1),
    yerr=train_errors.std(axis=1),
    label="Training error",
)
plt.errorbar(
    max_depth,
    test_errors.mean(axis=1),
    yerr=test_errors.std(axis=1),
    label="Testing error",
)
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 6: Validation Curve for Decision Tree using Train Errors",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Our target when running machine learning models is to get an unbiased estimate of our errors. This is possible when we understand how the concept of bias-variance tradeoff relates to underfitting and overfitting.

1.5 Bias-Variance Tradeoff

We can divide the prediction errors from machine learning algorithms into two:

- Reducible parts
- Irreducible parts.

The irreducible parts are caused by stochastic noise as a result of missing important features or measurement errors and can't be reduced even with very good models. The reducible part, on the other hand, constitutes errors due to:

- Bias
- Variance

Error due to Bias - Bias is the difference between our predicted values and the actual values we are trying to predict. These errors occur when the algorithm is too simple (less complex) to capture the functional relationship between our features. Models having high bias tend to oversimplify the model by not learning a lot from the training data. This causes the model to make some systematic mistakes, and therefore, the predictions will be biased. This is also referred to as underfitting.

Error due to Variance - We define variance as the measure of spread in our dataset. These errors occur when the algorithm is so complex that it extracts patterns from the noise in the dataset. Models with high variance perform very well on training data but do

not generalize well on unseen data. This is what we will refer to as overfitting, as discussed above.

So, we see two issues here:

1. If our model is too simple and has few parameters, then there is a tendency that we risk having a model with high bias and low variance.
2. If our model is too complex with many parameters, then it is highly likely the model will overfit on the dataset due to high variance and low bias on the dataset.

This is what introduces the bias-variance tradeoff since it is impossible to have a model that will be both complex and simple at the same time.

Building accurate models requires understanding these errors so as to avoid our model underfitting or overfitting.

In the next few cells, we will demonstrate the concept of bias and variance tradeoff.

```
In [ ]: from mlxtend.evaluate import bias_variance_decomp
```

We use the data as above and the same algorithm to demonstrate the concept of bias-variance tradeoff.

```
In [ ]: from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X, y = housing.data.values, housing.target.values * 10
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=1
)
```

In the next cell, we will define the hyperparameter that will define an increase in model complexity.

```
In [ ]: max_levels = list(range(1, 50))
levels = []
squared_bias_plus_variance = []
for level in max_levels:
    model = DecisionTreeRegressor(max_depth=level)
    model.fit(X_train, y_train)
    mse, bias, var = bias_variance_decomp(
        model,
        X_train,
        y_train,
        X_test,
        y_test,
        loss="mse",
        num_rounds=200,
        random_seed=1,
    )
    score = model.score(X_test, y_test)
    squared_bias_plus_variance.append(bias**2 + var)
    levels.append(level)
```

In the next cell, we plot the bias-variance tradeoff chart that shows how an increase in model complexity affects bias and variance.

```
In [ ]: import plotly.graph_objects as go

scatter = go.Scatter(x=levels, y=squared_bias_plus_variance)
layout = go.Layout(
    title="Fig. 7. Bias variance tradeoff",
    xaxis=dict(title="levels"),
    yaxis=dict(title="bias^2+variance"),
)
go.Figure(data=[scatter], layout=layout)
```

It can be seen that as the model complexity increases, the error reduces.

Now that we have a good understanding of how to improve our model performance, we can go ahead and learn how to tweak the hyperparameters of our models.

2. Hyperparameter Tuning

Hyperparameter tuning involves searching over a given set of values and finding the optimal values of a hyperparameter in a machine learning model. We expect that after performing hyperparameter tuning, the result of the model training will give us a better score without overfitting on the dataset.

Now let's highlight the difference between a parameter and a hyperparameter.

While a parameter is learned from the data during model training, a hyperparameter is set by the user before we begin training our model.

An example of a parameter is the coefficients we obtain from training a linear regression model while the number of epochs and batch size are examples of hyperparameters. We should note that some models like linear regression only have parameters while others like the K-Nearest Neighbors (KNN) contain only hyperparameters.

Before commencing on to hyperparameter tuning, we first define the **hyperparameter space**, which is the set of all possible hyperparameter values. Hyperparameters can either be discrete or continuous values. When defining the hyperparameter space, we also define their underlying distribution.

In the next subsections of this lesson, we intend to discuss the various types of hyperparameter tuning.

2.1 Manual Search

The manual search method entails defining the search space and looping all the possible hyperparameters to get a combination that will give us the best outcome. We follow these steps when performing manual search:

1. Split the train data into train and test splits.

2. Define the initial hyperparameter values.
3. Perform k-fold cross validation on the training data.
4. Get the validation score.
5. Choose another set of hyperparameter values.
6. Repeat steps 3-5 until we get satisfactory performance.
7. Train the model on the full train set with the chosen hyperparameter value.
8. Evaluate this performance on the test set.

In the example below, we show how we can get and set the value(s) of model hyperparameters. We will consider the [blood transfusion data](#).

```
In [ ]: from sklearn import set_config

set_config(display="diagram")

import pandas as pd
```

We load the data and display the first few rows.

```
In [ ]: url = "https://archive.ics.uci.edu/ml/machine-learning-databases/blood-transfusi
data_df = pd.read_csv(url)
data_df.head()
```

The next step involves renaming the columns for easier handling.

```
In [ ]: cols = [c.lower().split()[0] for c in data_df.columns]
cols[-1] = "class_name"
data_df.columns = cols
```

```
In [ ]: data_df.head()
```

We then define the target class that will set the stage for the type of machine learning problem we are supposed to solve. In our case, we are presented with a binary classification problem that we will not delve into too much as we are only interested in learning the hyperparameter tuning techniques.

```
In [ ]: target_name = "class_name"
feature_columns = ["recency", "frequency", "monetary"]

target = data_df[target_name]
data = data_df[feature_columns]
```

We transform the data by rescaling it using `StandardScaler` that removes the mean of the feature and scales to a unit variance.

```
In [ ]: from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model = Pipeline(
    steps=[
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
```

```
]
)
```

As discussed in the previous section, we evaluate the performance of the model using cross-validation.

```
In [ ]: from sklearn.model_selection import cross_validate

cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Accuracy score via cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

In the demonstrations that will follow in this lesson, we will try to tune the hyperparameters `learning_rate` and `max_leaf_nodes` of the `HistGradientBoostingClassifier` model. We start first by displaying their default values below.

```
In [ ]: print("learning_rate default value", model.get_params()["classifier__learning_ra
print("max_leaf_nodes default value", model.get_params()["classifier__max_leaf_n
```

Without specifying the value of the hyperparameters, the model uses the default parameters, for example, the default value for `C` is 1. We can manually tune the hyperparameter, in this case, change the value of `C` as shown in the cell below.

```
In [ ]: model.set_params(classifier__learning_rate=1e-3)
model.set_params(classifier__max_leaf_nodes=20)
cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Model accuracy score with cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

The hyperparameters are stored in `model.get_params` and we can explore them one by one. For demonstration purposes we will consider tuning the hyperparameter `C` in the cells below.

The cell below displays the values of the hyperparameters we have set above.

```
In [ ]: print("learning_rate default value", model.get_params()["classifier__learning_ra
print("max_leaf_nodes default value", model.get_params()["classifier__max_leaf_n
```

In the next cell, we manually set a couple of values for `C` and choose the one that gives us better performance.

```
In [ ]: for learning_rate in [1e-3, 1e-2, 1e-1, 1, 10]:
        for max_leaf_nodes in [3, 5, 10, 15, 20, 30]:
            model.set_params(classifier__learning_rate=learning_rate)
            model.set_params(classifier__max_leaf_nodes=max_leaf_nodes)
            cv_results = cross_validate(model, data, target)
            scores = cv_results["test_score"]
```

```
print(
    f"Model accuracy score with cross-validation:\n learning_rate={learn
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

From the cell above, we see that the value of `C=0.01` gives better accuracy.

2.2 Grid Search

Given a hyperparameter space, grid search looks for all the possible combinations that improve the performance of the model. In this method, the underlying distribution of the hyperparameters need not be known.

So, the steps involved in grid search are as follows.

1. Split the data into train and test sets.
2. Define the hyperparameter space.
3. Construct a nested loop of the number of hyperparameters.
4. Perform cross validation on the train set and store the resulting score in each loop.
5. Train the model on the train set and evaluate it on the test set.

The output of the grid search will be equal to the output of the manual search since the model is evaluated at each pair of hyperparameters in our grid.

Just like in the manual tuning section above, we demonstrate how to apply grid search in hyperparameter tuning.

We start by splitting our dataset into training and testing sets.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(data, target, random_state=4
```

We use the histogram gradient classifier model, which is a tree-based classifier.

```
In [ ]: from sklearn.ensemble import HistGradientBoostingClassifier

model = Pipeline(
    [
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)
model
```

Grid search is costly as it finds a combination of all the hyperparameters in our hyperparameter space. In this example, we share only limited nodes, and we choose to select only two hyperparameters to tune.

```
In [ ]: from sklearn.model_selection import GridSearchCV

param_grid = {
    "classifier__learning_rate": (0.001, 0.01, 0.1, 1, 10),
```

```

        "classifier__max_leaf_nodes": (3, 5, 10, 15, 20, 30),
    }
    model_grid_search = GridSearchCV(model, param_grid=param_grid, n_jobs=2, cv=2)
    model_grid_search.fit(X_train, y_train)

```

We then evaluate the accuracy of our model using the test set.

```

In [ ]: accuracy = model_grid_search.score(X_test, y_test)
        print(f"The test accuracy score of the grid-searched pipeline is: " f"{accuracy:

```

The grid search takes the hyperparameter space with all the hyperparameter values in it and finds all the possible combinations of the hyperparameters. Like in our case where we have four learning_rate and max_leaf_nodes values, there will be $4 \times 4 = 16$ combination of hyperparameters, and we select the pair that gives us the best result. This makes the process expensive because for any value we add, the process becomes more and more computationally unsustainable. We can actually see that Grid Search is similar to the manual search we saw in the previous subsection.

From the selected hyperparameters, we can now predict the first five targets.

```

In [ ]: model_grid_search.predict(X_test.iloc[0:5])

```

We now visualize the original values in the cell below.

```

In [ ]: y_test.iloc[0:5].values

```

By using the `best_params_` attribute, we can see the selected hyperparameters that gives us the highest accuracy.

```

In [ ]: print(f"The best set of parameters is: " f"{model_grid_search.best_params_}")

```

We inspect all the values stored in `cv_results_` using a dataframe as shown below.

```

In [ ]: cv_results = pd.DataFrame(model_grid_search.cv_results_).sort_values(
        "mean_test_score", ascending=False
    )
        cv_results.head()

```

We shorten the headers to make the dataframe more readable.

```

In [ ]: # get the parameter names
        column_results = [f"param_{name}" for name in param_grid.keys()]
        column_results += ["mean_test_score", "std_test_score", "rank_test_score"]
        cv_results = cv_results[column_results]

```

```

In [ ]: def shorten_param(param_name):
        if "__" in param_name:
            return param_name.rsplit("__", 1)[1]
        return param_name

        cv_results = cv_results.rename(shorten_param, axis=1)
        cv_results.head()

```

Since we are only interested in our two hyperparameters, the table below displays the model accuracy for each pair.

```
In [ ]: pivoted_cv_results = cv_results.pivot_table(
        values="mean_test_score", index=["learning_rate"], columns=["max_leaf_nodes"]
    )

pivoted_cv_results
```

We can visualize the table above in a heatmap as shown in the next cell.

```
In [ ]: import seaborn as sns

ax = sns.heatmap(pivoted_cv_results, annot=True, cmap="YlGnBu", vmin=0.0, vmax=0.9)
ax.invert_yaxis()
plt.suptitle(
    "Fig. 8: Heatmap of the Model Results",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

The biggest challenge we have with the two methods we have seen above is that they are very computationally intensive especially when our hyperparameter space is large. In the next subsection, we will discuss the random search method that tries to solve this problem.

2.3 Random Search

This method randomly selects the hyperparameter for each iteration. It outperforms the grid search when the data is small or we have no idea of the range of hyperparameter space. It is also much more efficient when compared to grid search. In random search, the hyperparameter distribution must be specified together with the hyperparameter space, which is wider as compared to the grid search case.

The steps involved in random search are:

1. Split the data into train and test sets.
2. Define the number of trials and set a random seed.
3. Define the hyperparameter space with its distribution.
4. Define an iterator consisting of random hyperparameter combinations.
5. Loop through the iterator.
6. After getting the best possible combination of hyperparameters, we train the model on the full train set and evaluate the model performance.

In the following cells, we demonstrate the random search method for tuning the hyperparameters on our blood transfusion data.

```
In [ ]: model = Pipeline(
    [
        ("preprocessor", StandardScaler()),
```

```

        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)

```

We can view the default hyperparameter values in the cell below.

```

In [ ]: from pprint import pprint

print("Default parameters:\n")
pprint(model.get_params())

```

The next cell defines the hyperparameter search space.

```

In [ ]: from scipy.stats import loguniform

class loguniform_int:
    """Integer valued version of the log-uniform distribution"""

    def __init__(self, a, b):
        self._distribution = loguniform(a, b)

    def rvs(self, *args, **kwargs):
        """Random variable sample"""
        return self._distribution.rvs(*args, **kwargs).astype(int)

from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "classifier__learning_rate": loguniform(1e-6, 1e3),
    "classifier__max_leaf_nodes": loguniform_int(3, 100),
}

```

We can now train the model on our training data using cross-validation that executes 100 iterations of 5 folds.

```

In [ ]: model_random_search = RandomizedSearchCV(
    model,
    param_distributions=param_distributions,
    n_iter=10,
    cv=5,
    verbose=1,
    n_jobs=-1,
    return_train_score=True,
)
model_random_search.fit(X_train, y_train)

```

The random search method for hyperparameter tuning selects the hyperparameter combinations randomly as opposed to grid search where each value in our hyperparameter space was used explicitly. In random search, the choice of one hyperparameter value is independent from the other ones. As opposed to grid search, where our hyperparameter space consisted of hyperparameter values, random search allows the use of sampling distributions.

The next cell displays the best parameters from our model.

```
In [ ]: model_random_search.best_params_
```

We evaluate our model performance on the test dataset; however, we do not expect good results as the hyperparameters were arbitrarily chosen for the purposes of demonstrating the random search method.

```
In [ ]: accuracy = model_random_search.score(X_test, y_test)

print(f"The test accuracy score of the best model is " f"{accuracy:.3f}")
```

Just like the grid search process above, we inspect the cv results in the next cell.

```
In [ ]: cv_results_rs = pd.DataFrame(model_random_search.cv_results_).sort_values(
        "mean_test_score", ascending=False
    )
cv_results_rs.head()
```

```
In [ ]: def shorten_param(param_name):
        if "__" in param_name:
            return param_name.rsplit("__", 1)[1]
        return param_name

cv_results_rs = cv_results_rs.rename(shorten_param, axis=1)
cv_results_rs.head()
```

The table below displays the pairs that the random search tuning picked by the random search algorithm. Since `n_iter=10`, random search will pick 10 pairs at random.

```
In [ ]: pivoted_cv_results_rs = cv_results_rs.pivot_table(
        values="mean_test_score", index=["learning_rate"], columns=["max_leaf_nodes"
    ])

pivoted_cv_results_rs
```

We can observe that hyperparameter tuning using random search does not evaluate the model at all pairs but randomly selects the best possible combination among pairs. Since random search does not pick all possible pairs, it tends to be faster than manual search and grid search, and its accuracy is nearly equal to grid search.

2.4 Bayesian Optimization

Bayesian optimization improves search space based on what it learns from the previous iterations. In this kind of hyperparameter selection, we first build the probability model of the objective function and then choose the optimal hyperparameters.

There are several techniques of the Bayesian Optimization listed below.

- Gaussian Process (GP).
- Sequential model-based optimization
- Tree Parzen Estimator (TPE)

Bayesian optimization memorizes the model's past performance, which it uses to find the probability of a score given a parameter, that is, $P(\text{score}|\text{hyperparameters})$.

The model can be optimized easily and used to find the next hyperparameter(s).

The steps of a Bayesian optimization are:

1. Find the probability model of a score given a hyperparameter.
2. Find the optimal hyperparameters of the model in (1).
3. Plug the hyperparameters into the objective function.
4. Update the probability model given the new results.
5. Repeat steps to 1 to 4 until we get a satisfactory result.

We will discuss more on Bayesian optimization in lesson 4.

3. Conclusion

In this lesson, we have seen the various techniques for improving model performance and the impact of hyperparameters on model performance. We have also discussed how to automate the search for hyperparameters using grid search and randomized search. In the next lesson, we will apply these methods to a real-world example.

References

1. Badillo, Solveig, et al. "An Introduction to Machine Learning." *Clinical Pharmacology & Therapeutics*, vol. 107, no. 4, 2020, pp. 871-885.
2. Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.
3. Yang, Li, and Abdallah Shami. "On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice." *Neurocomputing*, vol. 415, 2020, pp. 295-316.